

Documentación del Proyecto de Compiladores (m2r)

Nick Hernd

17 de junio de 2026

Índice general

1. Introducción	2
2. Arquitectura del Compilador	3
2.1. Análisis Léxico y Sintáctico	3
2.2. Análisis Semántico y Generación de Código	3
2.3. Máquina Virtual m2r	4
3. Manual de Usuario	5
3.1. Compilación	5
3.2. Ejecución	5
3.3. Depuración en la Máquina Virtual	5

Capítulo 1

Introducción

El proyecto *m2r* es una herramienta educativa diseñada para la enseñanza de los principios fundamentales de la compilación de lenguajes de programación. Su objetivo principal es permitir al estudiante comprender el proceso de traducción de un lenguaje de alto nivel a un lenguaje intermedio ejecutable por una máquina virtual.

Este sistema se divide en dos componentes principales:

1. **El Compilador:** Encargado de realizar el análisis léxico, sintáctico y semántico del código fuente, generando un código intermedio.
2. **La Máquina Virtual (Intérprete m2r):** Encargada de ejecutar el código generado por el compilador, simulando un entorno de ejecución con memoria, registros y un ciclo de instrucción definido.

Este documento aborda los fundamentos teóricos detrás del diseño de compiladores, el detalle técnico de la implementación de este proyecto, y una guía práctica para su uso y extensión.

Capítulo 2

Arquitectura del Compilador

La arquitectura del sistema sigue el modelo clásico de compilación y ejecución de programas.

2.1. Análisis Léxico y Sintáctico

El compilador utiliza *Flex* y *Bison* para la generación automática de los analizadores:

- **Flex (src/lexer.l):** Define los patrones léxicos del lenguaje (tokens). Convierte el flujo de caracteres de entrada en una secuencia de tokens.
- **Bison (src/parser.y):** Define la gramática libre de contexto del lenguaje. Construye un Árbol de Sintaxis Abstracta (AST) que representa la estructura jerárquica del programa.

2.2. Análisis Semántico y Generación de Código

Una vez construido el AST, el compilador realiza:

1. **Tabla de Símbolos (src/TablaSimbolos.cc):** Gestiona el ámbito y el tipo de los identificadores declarados.
2. **Tabla de Tipos (src/TablaTipos.cc):** Verifica la consistencia de los tipos en las expresiones (verificación de tipos).
3. **Generador de Código:** Recorre el AST para emitir las instrucciones de bajo nivel compatibles con la VM *m2r*.

2.3. Máquina Virtual m2r

El intérprete (`m2r.c`) emula una arquitectura basada en:

- **Memoria:** Un array de estructuras `MEMO` que almacenan enteros o reales.
- **Registros:** Acumulador y Registro Base.
- **Ciclo de Instrucción:** *Fetch-Decode-Execute*. Lee la instrucción en el PC (Program Counter), decodifica la operación y sus operandos, y ejecuta la acción correspondiente (aritmética, saltos, E/S).

Capítulo 3

Manual de Usuario

Este manual proporciona las instrucciones necesarias para compilar, ejecutar y depurar programas utilizando el sistema *m2r*.

3.1. Compilación

Para compilar el proyecto, asegúrese de tener `make`, `gcc`, `g++`, `flex` y `bison` instalados. En la raíz del proyecto, ejecute:

```
make
```

Esto generará los ejecutables `compiler` y `m2r`.

3.2. Ejecución

El proceso de ejecución se realiza en dos etapas:

1. **Compilación del código fuente:**

```
./compiler <archivo_fuente.java>
```

Esto genera un archivo de código intermedio (normalmente `output.asm`).

2. **Interpretación:**

```
./m2r output.asm
```

3.3. Depuración en la Máquina Virtual

El intérprete `m2r` incluye un depurador básico. Para activarlo (suponiendo que la implementación lo permita mediante banderas o configuraciones), se pueden utilizar comandos interactivos durante la ejecución para:

- Listar instrucciones.
- Establecer puntos de ruptura (*breakpoints*).
- Inspeccionar el contenido de la memoria y registros (Acumulador, Base).
- Ejecutar paso a paso (*step-by-step*).

Consulte el código fuente de `src/m2r.c` para conocer los comandos específicos de depuración soportados.